

2강

MVC패턴





spring

➤ 디자인 패턴

- 소프트웨어 설계 시 자주 발생하는 문제를 해결하기 위한 일반적인 해결 방법을 정리한 것
- 개발 과정에서 반복적으로 나타나는 문제들을 효과적으로 해결할 수 있도록 정형화된 설계 기법

➤ 디자인 패턴의 필요성

- 코드 재사용성 증가 → 검증된 해결책을 재사용하여 효율적인 개발 가능
- 유지보수성 향상 → 구조적으로 짜여 있어 유지보수와 수정이 용이
- 가독성 및 협업 강화 → 팀원 간 코드 이해가 쉬워지고, 협업이 원활해짐
- 소프트웨어 품질 향상 → 안정적이고 유연한 코드 설계 가능
- 디자인 패턴을 활용하면 코드 품질을 향상시키고,

✓ 디자인 패턴을 활용하면 코드 품질을 향상시키고, 유지보수를 쉽게 만들며,
협업을 효율적으로 할 수 있습니다!



spring

➤ • MVC 디자인 패턴이란?

- Model - View - Controller 구조로 프로그래밍하는 설계 방법
- 'MVC 패턴' 또는 'MVC 디자인 패턴' 이라고 함
- 웹 프로그래밍뿐만 아니라 다양한 소프트웨어 개발에서 사용됨





spring

➤ MVC 패턴 구성 요소

- Model (모델): 데이터베이스와 연결, 비즈니스 로직 처리
- View (뷰): 화면(UI) 구성, 사용자에게 결과 출력
- Controller (컨트롤러): 사용자 요청을 받아 Model과 View 연결

➤ MVC 패턴의 장점

- 프레젠테이션 로직(View)과 비즈니스 로직(Model) 분리
- 모듈화: 개발이 체계적이고 유지보수 용이
- 새로운 개발자가 전체 구조를 쉽게 이해 가능

✓ Spring MVC 프레임워크 역시 MVC 패턴을 기반으로 동작!

SPRING MVC

프레임워크





spring

➤ 스프링 MVC

- 웹 애플리케이션 개발을 위한 프레임워크
- MVC 패턴을 기본으로 해서 개발을 체계적이고 쉽게 진행할 수 있도록 설계됨

➤ Spring MVC 흐름

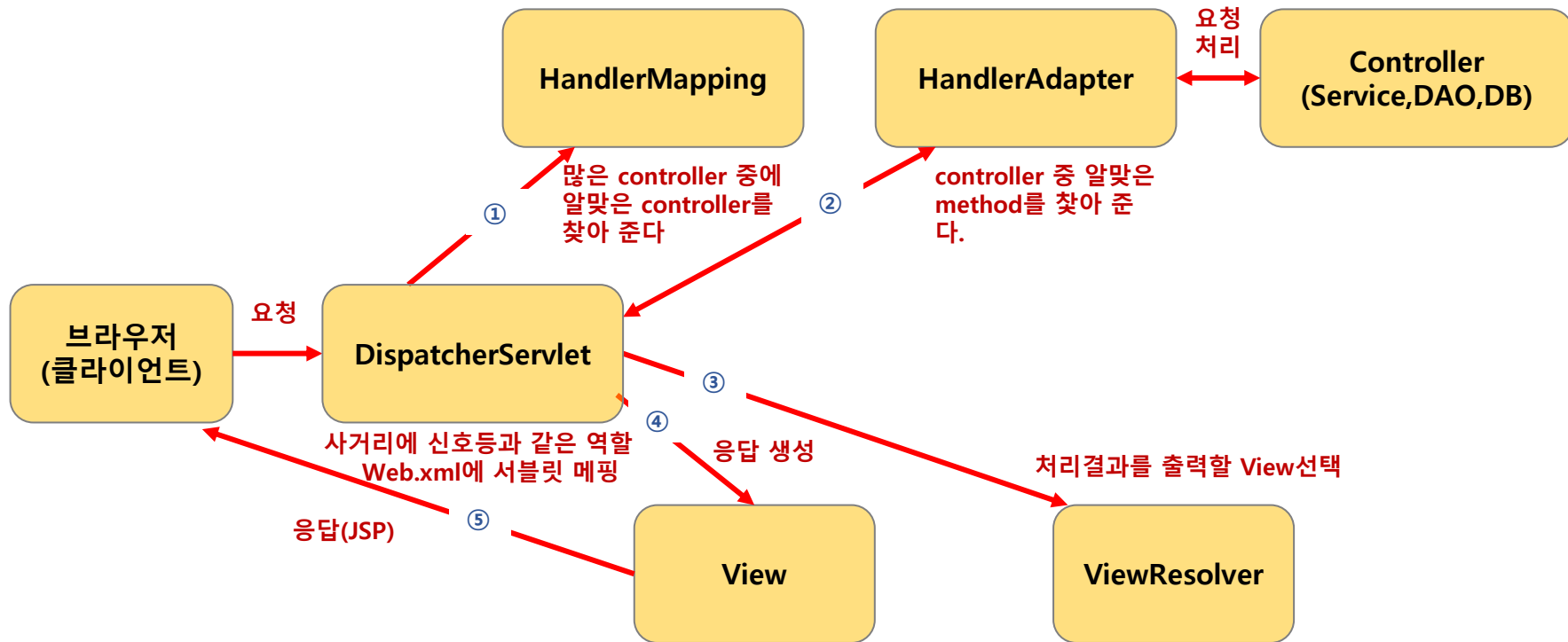
- 클라이언트가 요청을 보면, DispatcherServlet이 요청을 받고, 적절한 Controller로 전달
- Controller가 Model을 처리하고, View를 선택
- View가 클라이언트에게 결과를 출력

✓ 스프링 MVC를 학습할 때, MVC 패턴을 머릿속에 그려두고 이해하면 더 쉽게 구조를 파악할 수 있습니다.



spring

➤ 스프링 MVC





➤ 스프링 MVC

- DispatcherServlet: 클라이언트의 요청을 분석해서 가장 적합한 핸들러(Controller method)를 찾음
- HandlerMapping: 클라이언트 요청에 적합한 핸들러(Controller method) 반환
- HandlerAdapter: 클라이언트 요청에 적합한 핸들러(Controller method) 호출
- ViewResolver: 응답에 적합한 뷰(view)를 찾음
- MVC 패턴에서 Model은 비즈니스 로직을 수행하는 계층을 의미하지만, Spring에서는 실제로 이 역할을 Service 계층이 담당



spring

➤ 스프링 MVC 프레임워크를 사용하는 이유

스프링 MVC의 장점

- 복잡한 설정과 구조는 이미 스프링 MVC가 준비 했기 때문에 개발자는 Controller와 View만 구현하면 됨
- DispatcherServlet, HandlerMapping, HandlerAdapter 등은 이미 스프링 MVC 프레임워크 내에 내장되어 있어, 개발자는 이를 신경 쓸 필요가 없음

왜 프레임워크를 사용하는가?

- 프레임워크 사용의 이유는 구조가 이미 마련되어 있기 때문
- 개발자는 구조를 이해하고, Controller와 View만 집중해서 개발하면 됨
- 복잡한 웹 애플리케이션 구조를 직접 만들지 않아도 되는 이점이 있음

구조를 이해하는 중요성

- 전체적인 구조를 정확히 이해해야 개발에 무리가 없음
- 스프링 MVC 프레임워크의 구조를 정확히 파악하고 개발을 진행하는 것이 중요